

Software Architecture and Design Specification

Student Attendance Management System

Item	Details
Project	Student Attendance Management System
Version	1.0
Authors	Bindu S, Sohan M Rajannavar, Srinikesh Durgavajjula, Taanya Gupta
Date	09-09-2026
Status	Review

Revision History

Version	Date	Author	Change Summary
1.0	09-09-2026	<Team>	Initial architecture and design specification prepared.

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof Niveditha Reddy	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Student Attendance Management System (SAMS), including system structure, components, interfaces, security, data flow, and design decisions.

1.2 Scope

The system covers secure authentication, student and faculty management, subject/class management, attendance sessions, attendance marking and correction, attendance history, percentage calculation, reports, low-attendance identification, and administration.

1.3 Audience

Developers, QA engineers, project guides, administrators, faculty users, security reviewers, and maintenance teams.

1.4 Definitions

SAMS, Admin, Faculty, Student, RBAC, REST API, DB, Attendance Session, Attendance Percentage, RTM, SRS, and STP.

2. Document Overview

2.1 How to use this document

This document provides the architectural and design deliverables for SAMS, including component structure, UML diagrams, design decisions, security considerations, API definitions, and traceability.

2.2 Related Documents

Student Attendance Management System SRS, Software Test Plan (STP), Requirements Traceability Matrix (RTM), and project database/design documentation.

3. Architecture

3.1 Goals & Constraints

Goals: Secure authentication, reliable attendance recording, accurate percentage calculation, maintainable modular design, usability, and controlled role-based access.

Constraints: Institutional attendance rules, available resources, database consistency, network availability, and project scope/time limitations.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Students	Accurate attendance records, easy viewing, privacy.
Faculty	Fast attendance marking, correction, history, reports.
Administrators	User management, configuration, auditability, security.
Project Team	Modularity, maintainability, testability.
Project Guide / Evaluators	Correct requirements coverage and reliable operation.

3.3 Component (UML) Diagram

The following UML component diagram represents the main actors, application modules, and database interactions of the Student Attendance Management System.

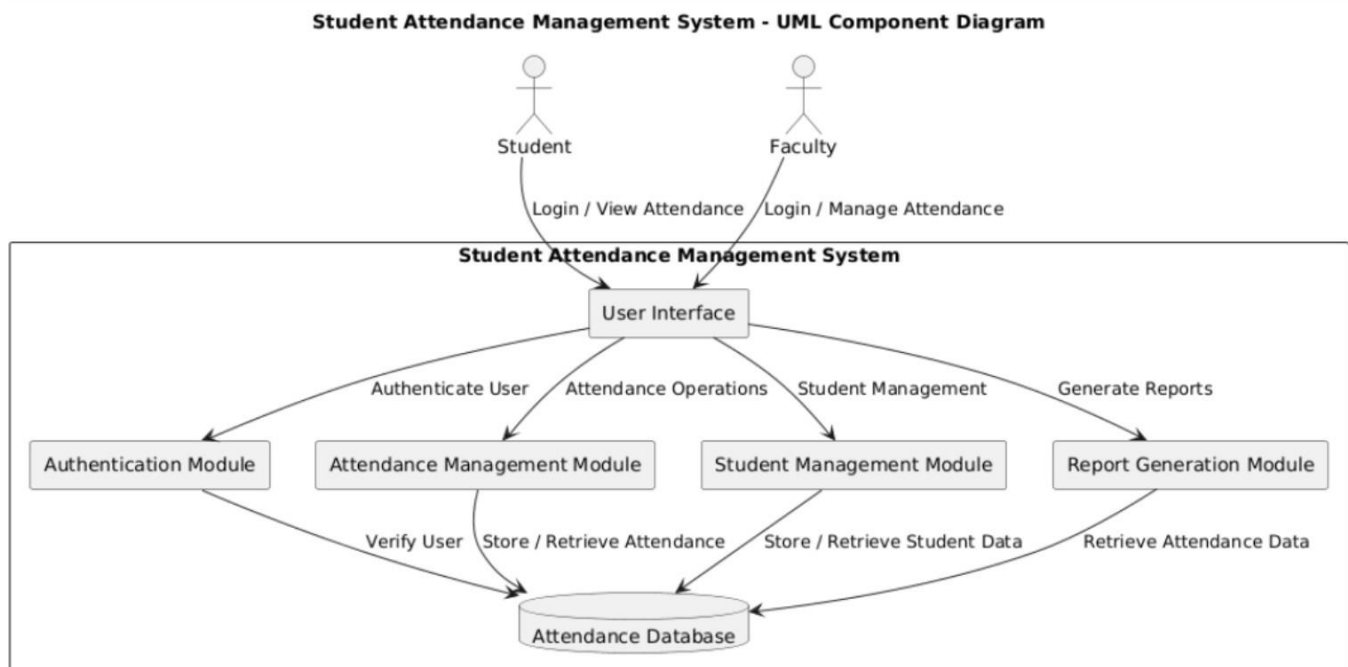


Figure 1. UML Component Diagram – Student Attendance Management System

3.4 Component Descriptions

Component	Description
User Interface	Screens for login, dashboards, attendance, records, and reports.
Authentication & RBAC	Authenticates users and controls Admin, Faculty, and Student permissions.
Student Management	Creates, updates, searches, activates/deactivates, and retrieves student records.
Faculty & Subject Management	Manages faculty accounts, subjects, classes, and assignments.

Attendance Management	Creates sessions, marks attendance, validates updates, and stores attendance records.
Reporting & Analytics	Calculates percentages, generates reports, and identifies low-attendance students.
Notification / Alert	Supports low-attendance alerts where enabled.
Database	Stores users, students, faculty, subjects, classes, sessions, attendance, and audit records.

3.5 Chosen Architecture Pattern and Rationale

A three-layer/layered architecture is chosen: Presentation Layer, Application/Business Logic Layer, and Data Access Layer. It provides separation of concerns, simplifies testing and maintenance, and is suitable for a college attendance application. Microservices are not selected because they add unnecessary deployment and operational complexity for this project scope.

3.6 Technology Stack & Data Stores

Layer	Proposed Technology
Frontend	HTML/CSS/JavaScript or project-approved framework
Backend	Python/Java/Node.js based on final implementation
API	RESTful HTTP/HTTPS APIs
Database	MySQL or another approved relational database
Version Control	Git and GitHub
Security	Password hashing, session/token security, HTTPS where deployed

3.7 Risks & Mitigations

Risk	Mitigation
Incorrect attendance calculation	Centralize calculation logic and test boundary cases.
Unauthorized attendance changes	Enforce RBAC and server-side authorization.
Database failure/data loss	Use backups and recovery procedures.
Network interruption	Provide clear error handling and safe retry/reload behavior.
Duplicate attendance records	Use database constraints and server-side validation.

3.8 Traceability to Requirements

Requirement	Architecture Component
SAMS-F-001: Login/RBAC	Authentication & RBAC Service
SAMS-F-006: Mark attendance	Attendance Management Service
SAMS-F-007: Update attendance	Attendance Management + Audit Service
SAMS-F-009: Attendance percentage	Reporting & Analytics Service
SAMS-F-010: Attendance reports	Reporting & Analytics Service
SAMS-F-011: Low attendance	Reporting + Notification Service
SAMS-SR-001 to SR-006	Authentication, RBAC, API, Database, Audit components

3.9 Security Architecture

- Authentication: verify user identity before access.
- Authorization: enforce Admin, Faculty, and Student permissions on the server.
- Data protection: use password hashing and HTTPS/TLS where applicable.
- Input validation: validate IDs, attendance values, dates, and request parameters.
- Auditability: record important attendance and administrative changes.
- Session security: expire or invalidate sessions on logout.

4. Design

4.1 Design Overview

SAMS is designed as a modular layered application. Requests move from the interface to application services, which validate permissions and business rules before accessing the database. Attendance calculations and reports are performed centrally for consistent results.

4.2 UML Sequence Diagrams

The following sequence diagrams describe two important system workflows: marking attendance and generating an attendance report.

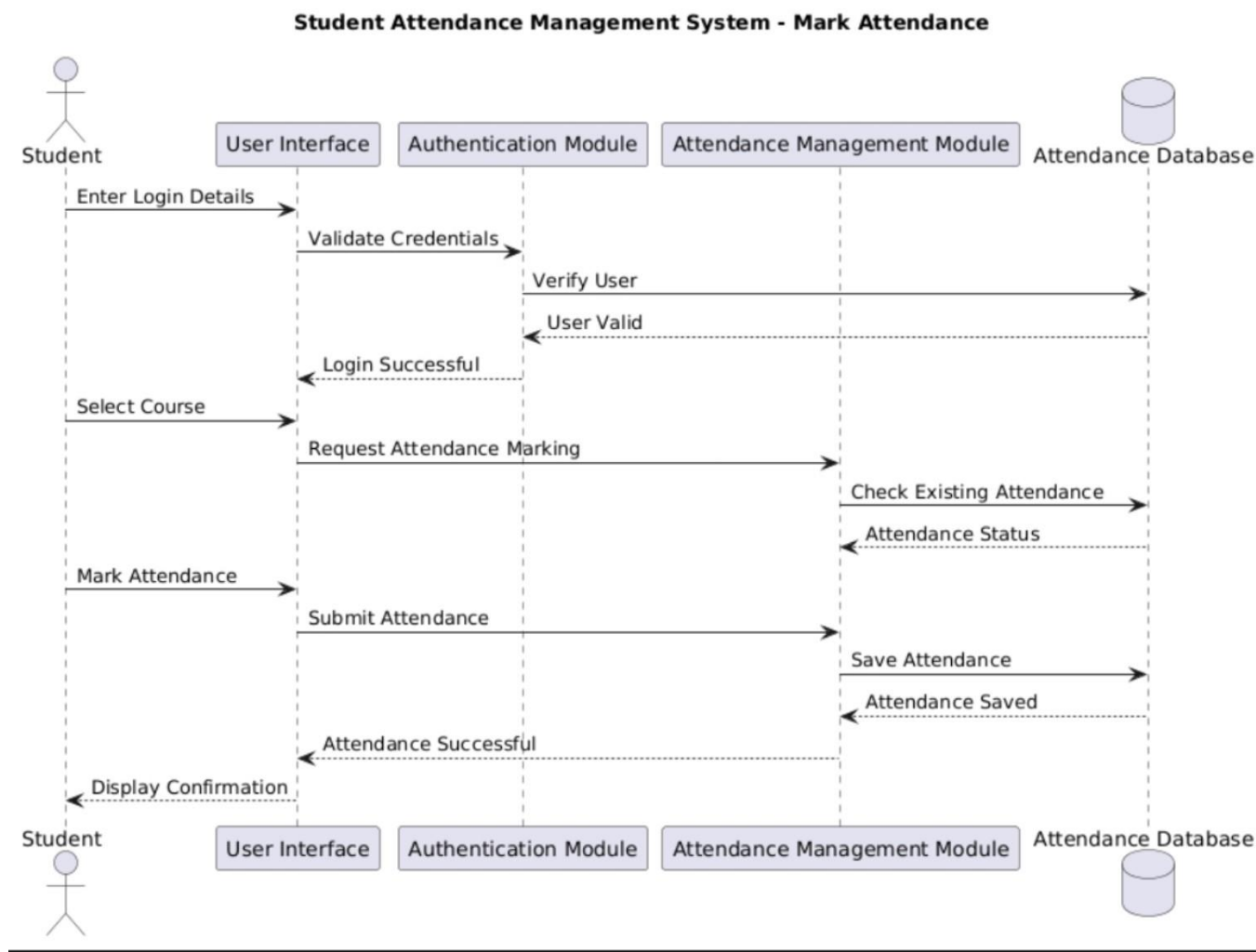


Figure 2. UML Sequence Diagram – Mark Attendance

4.3 API Design

Endpoint	Method	Request	Response / Errors
----------	--------	---------	-------------------

/api/auth/login	POST	{username, password}	{token, role}; 401 Invalid credentials
/api/attendance/mark	POST	{studentId, sessionId, status}	{status}; 400 Invalid data, 403 Unauthorized
/api/attendance/percentage	GET	studentId / subjectId	{percentage, attended, conducted}; 404 Not found
/api/attendance/report	GET	classId, subjectId	Attendance report; 403 Unauthorized

4.4 Error Handling, Logging & Monitoring

Use standardized errors for invalid credentials, unauthorized actions, invalid data, duplicate attendance, missing records, and database failures. Passwords and sensitive information must not be logged. Monitor login failures, API errors, attendance update failures, database connectivity, and application availability.

4.5 UX Design

Provide clear role-based dashboards, readable tables, simple attendance controls, search/filter options, validation messages, responsive layouts, and accessible font sizes. Students should have a simple subject-wise attendance and percentage view.

4.6 Open Issues & Next Steps

- Finalize the technology stack.
- Finalize the UML component and sequence diagrams.
- Finalize database schema and API contracts.
- Implement and test security controls.
- Future enhancements may include mobile application, biometric/RFID integration, analytics, and automated notifications.

5. Appendices

5.1 Glossary

Term	Meaning
SAMS	Student Attendance Management System
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SRS	Software Requirements Specification
STP	Software Test Plan
TLS	Transport Layer Security
UAT	User Acceptance Testing

5.2 References

- Student Attendance Management System SRS v1.0
- Student Attendance Management System STP v1.0
- IEEE software architecture/design guidance
- OWASP application security guidance

5.3 Tools

draw.io, PlantUML, GitHub, database management tools, Postman or equivalent API testing tools, and the IDE/toolchain selected by the project team.